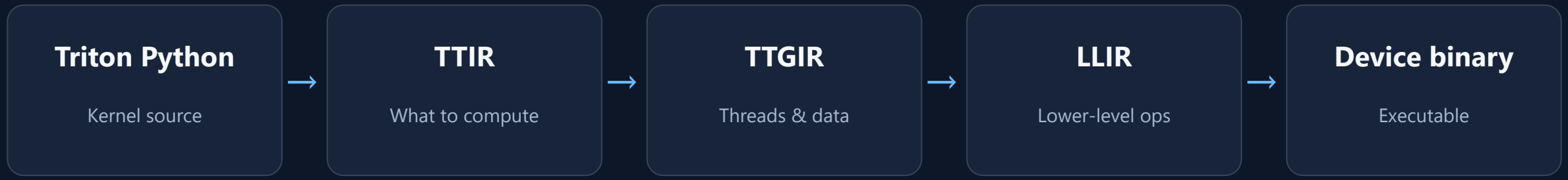
From Triton to GPU instructions

Today: understand the compilation pipeline



Two kernels · three configuration parameters · correctness first

Two kernels, two patterns

Matmul / GEMM

$$C = A \times B$$

Tiled multiplication + accumulation

Softmax

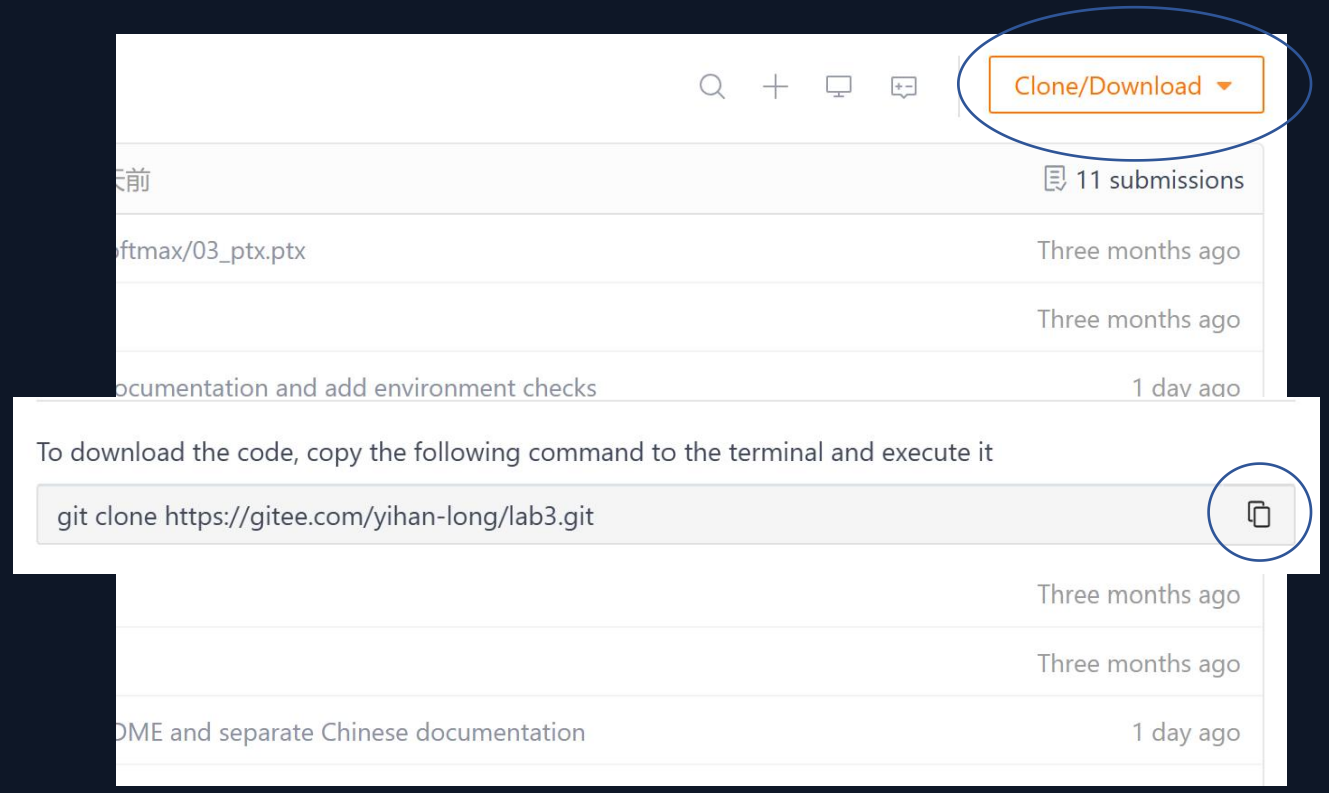
$$\exp(x - \max(x)) / \Sigma \exp(x - \max(x))$$

Elementwise operations + reductions

Clone the repository

- Open the Gitee repository
- Copy the clone command and run it
- Enter lab3 and confirm the files exist

gitee.com/yihan-long/lab3



Verify the environment

- Run the environment checker from lab3
- Confirm GPU, backend and numerical checks pass
- No new dependencies should be needed

```
root@423853f81753:~/lab3# python3 check_env.py
Python: /usr/local/bin/python3
Version: 3.12.11
CUDA_VISIBLE_DEVICES: None
[PASS] ixsmi: GPU  0: Iluvatar BI-V150 (UUID: GPU-c629ad24-4a8b-51f8-9a26-d436eae21edb)
[PASS] torch: version=2.10.0; path=/usr/local/lib/python3.12/site-packages/torch/__init__.py
[PASS] triton: version=3.6.0; path=/usr/local/lib/python3.12/site-packages/triton/__init__.py
[PASS] numpy: version=1.26.4; path=/usr/local/lib/python3.12/site-packages/numpy/__init__.py
[PASS] GPU runtime: available=True; logical_device_count=1; torch.version.cuda=10.2
[PASS] BI-V150 device: index=0; name=Iluvatar BI-V150; memory=32.0 GiB
[PASS] run_lab runtime gate: torch.version.cuda=10.2; the current runner requires a non-None value.
[PASS] Triton target: GPUTarget(backend='corex', arch=71, warp_size=64)
[PASS] warp size: 64
[PASS] matmul compile and correctness: shape=127x95x65; max_abs_err=4.96395e-06; atol=1e-3; rtol=1e-3
[PASS] matmul artifact stages: asm, cubin, llir, source, ttgir, ttir
[PASS] softmax compile and correctness: shape=17x1003; max_abs_err=2.78468e-09; atol=1e-6; rtol=1e-5
Environment and kernel smoke checks passed.
```

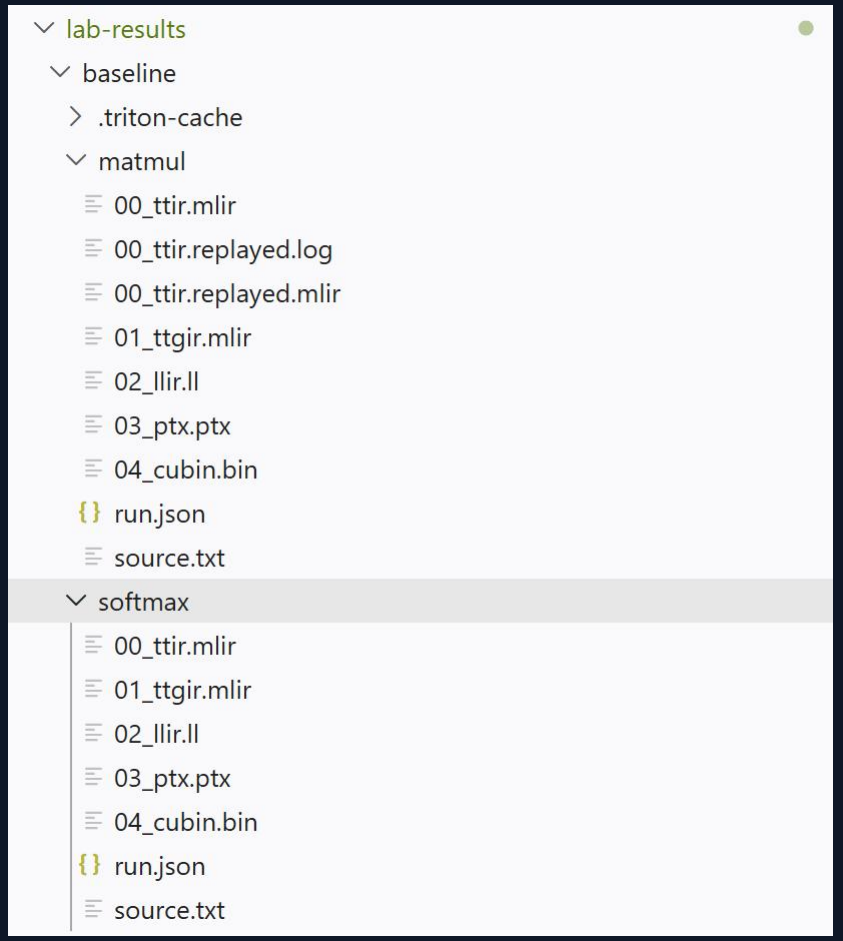
```
python3 check_env.py
```

Run the baseline

- Create the results directory as shown in README
- Run both existing kernels
- Keep the output for later comparisons

```
python3 run_lab.py --kernel all --device-index 0 --dump-dir "$V150_RUN_DIR/baseline" 2>&1 | tee "$V150_RUN_DIR/baseline.log"

cat "$V150_RUN_DIR/baseline/matmul/run.json"
cat "$V150_RUN_DIR/baseline/softmax/run.json"
```



TTIR: what are we computing?

Softmax

Load → Max → Subtract
→ Exp → Sum → Divide → Store

Matmul

Program ID → Load → K loop
→ Dot → Store

Find the main operations in the newly generated TTIR; exact code can differ.

TTGIR: how is work distributed?

- blocked: distribute tensor elements across threads
- shared: CTA(thread block)-local temporary storage
- matrix operand layouts: prepare data for dot

```
#smem = #ttg.shared_memory
#loc43 = loc("a_ptr"(#loc))
#loc44 = loc("b_ptr"(#loc))
#loc45 = loc("c_ptr"(#loc))
module attributes {"ttg.num-ctas" = 1 : i32, "ttg.num-warps" = 4 : i32, ttg.tar
  tt.func public @matmul_kernel(%a_ptr: !tt.ptr<f16> {tt.divisibility = 16 : i3
    %cst = arith.constant dense<0.000000e+00> : tensor<64x64xf32, #mma> loc(#lo
    %c0_i32 = arith.constant 0 : i32 loc(#loc1)
    %c512_i32 = arith.constant 512 : i32 loc(#loc1)
    %c32_i32 = arith.constant 32 : i32 loc(#loc1)
    %c8_i32 = arith.constant 8 : i32 loc(#loc1)
    %cst_0 = arith.constant dense<0.000000e+00> : tensor<32x64xf16, #blocked> l
    %cst_1 = arith.constant dense<0.000000e+00> : tensor<64x32xf16, #blocked1>
    %c64_i32 = arith.constant 64 : i32 loc(#loc1)
    %cst_2 = arith.constant dense<512> : tensor<64x1xi32, #blocked1> loc(#loc1)
    %cst_3 = arith.constant dense<512> : tensor<64x1xi32, #blocked2> loc(#loc1)
```

Example layout

1 warp = 64 threads on BI-V150

Pipeline and low-level IR

Software pipelining

Prepare future tiles while computing the current tile

num_stages = buffering depth

LLIR

Low-level operations for the active GPU backend

Exact instructions may differ by target

Hands-on: compare configurations

Tile size

BLOCK_M / BLOCK_N / BLOCK_K

Tile dimensions & reuse

Warp count

num_warps = 2 / 4 / 8

Thread participation & layout

Pipeline depth

num_stages = 1 / 3 / 5

Buffering & prefetch

Run the provided README commands; compare each configuration with baseline.

Validate results & takeaways

- Did the command run or report a useful error?
- Are the numerical results acceptable?
- What changed in the IR or benchmark?

TTIR

What is computed?

TTGIR

Who computes, where data lives?

LLIR

Lower-level GPU operations

Correctness and interpretation matter more than speedups.